

An engineer asks your agent to add comprehensive tests to a legacy codebase with 200 files and minimal existing test coverage. The engineer hasn't specified which modules to prioritize. How should the agent decompose this open-ended task?

- ☒ **A** Use Glob and Grep to map codebase structure, identify heavily-coupled modules, create a prioritized plan for high-impact areas, and revise as dependencies are discovered.
- ☐ **B** Systematically read all 200 files to create a complete function inventory before writing any tests, ensuring the testing plan accounts for every function before beginning.
- ☐ **C** Create a fixed testing schedule upfront based on directory structure, allocating equal effort to each top-level directory regardless of code complexity or business importance.
- ☐ **D** Start writing tests for the first module alphabetically, using test failures and imports to discover related files organically.

An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration. What's the most effective approach?

- ☐ **A** Launch a fresh subagent and include the prior transcript in the initial prompt for context.
- ☐ **B** Resume the subagent from its previous transcript without mentioning the changes—the architecture understanding remains valid.
- ☐ **C** Resume the subagent from its previous transcript and inform it about the renamed functions.
- ☒ **D** Launch a fresh subagent with a summary of prior findings.

An engineer sees an unfamiliar error message "SYNC_CONFLICT: entity version mismatch detected" in production logs but doesn't know which of the 12 services in the codebase generates it. They ask the agent to help locate the source code. What exploration approach will most efficiently find the responsible code?

- ☒ **A** Use Grep to search for distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context.
- ☐ **B** Use Grep to find all files that import the project's error handling module, then Read those files to locate custom error definitions.
- ☐ **C** Read the project's README and service configuration files to understand the architecture, then systematically Read source files in each service directory.
- ☐ **D** Use Glob to find files in directories commonly associated with error handling (such as `errors/`, `exceptions/`, or `handlers/`) across all services, then Read each matching file.

An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in-place. Today, they want to explore both approaches in depth—having the agent propose specific code changes for each—before deciding which to implement. What's the most effective way to structure this exploration?

- ☒ **A** Use `fork_session` to create two branches from yesterday's analysis, exploring one approach in each fork.
- ☐ **B** Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context.
- ☐ **C** Resume yesterday's session and explore both approaches sequentially within the same conversation thread.
- ☐ **D** Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context.

Your productivity agent connects to three MCP servers: an issue tracker (`search_issues`, `get_issue`, `create_comment`), a documentation wiki (`search_docs`, `get_page`, `list_spaces`), and a database explorer (`run_query`, `get_schema`, `list_databases`). When engineers ask cross-system questions like "What database tables are affected by the authentication refactor in PROJ-1234?", monitoring shows the agent makes 8-10 sequential tool calls, frequently issues exploratory calls because it lacks visibility into what content each server contains, and exhausts context space before completing complex investigations. What architectural change best leverages MCP capabilities to address these issues?

- ☐ **A** Add a `prepare_investigation` tool to each server that accepts a natural language question and returns relevant content summaries
- ☐ **B** Consolidate all three servers into a unified MCP server with cross-referencing capabilities
- ☐ **C** Add an orchestrator that routes questions to a single server based on keywords
- ☒ **D** Expose each server's content catalog as MCP resources—issue summaries, documentation hierarchy, database schemas

An engineer asks the agent to find all files in the monorepo that import the `@company/auth` package to understand how authentication is used across services. Which built-in tool is most appropriate for this task?

- ☐ **A** Glob, to find files with "auth" in their filename or path
- ☐ **B** Read, starting with package.json files to trace dependency declarations
- ☒ **C** Grep, to search for the import statement pattern across file contents
- ☐ **D** Bash, to execute `find . -type d -name "*auth*"` and explore matching directories

An engineer submits two requests:

- Request A: "Rename the `getUserData` function to `fetchUserProfile` everywhere it's used."
- Request B: "Improve error handling throughout the data processing module—add try/catch blocks, meaningful error messages, and ensure failures don't silently corrupt data."

For which request does specifying an explicit multi-phase workflow (such as analyze → propose → implement with review) most improve outcome quality?

☐ **A** Neither request benefits significantly

☐ **B** Both requests benefit equally

☒ **C** Request B, the error handling task

☐ **D** Request A, the function rename task

Your agent needs to insert a new helper function into the middle of a 150-line utility module, between two existing functions. The Edit tool fails because its `old_string` parameter cannot find unique text to match — the file has repetitive docstrings, variable names, and structural patterns. What's the most reliable way to complete this insertion?

- ☐ **A** Use Bash to append the function definition to the end of the file using heredoc syntax
- ☒ **B** Use Read to load the file, add the function at the appropriate location, then Write the updated file
- ☐ **C** Use Edit with an extremely long `old_string` capturing 30+ lines of context to guarantee uniqueness
- ☐ **D** Use Edit's `replace_all` parameter to target a common pattern and embed the new function in the replacement text

An engineer who just joined the team asks the agent to help them understand the authentication and authorization architecture before making security improvements. The codebase has 800+ files across multiple services. What exploration strategy will most effectively build understanding, given Claude built-in tools and context limits?

- ☒ **A** Use Grep to find authentication entry points, read those files, then follow imports and function calls to map the auth flow incrementally.
- ☐ **B** Launch parallel subagents to explore different services simultaneously, then synthesize their findings into an architectural overview.
- ☐ **C** Read any CLAUDE.md and README files first, then ask the engineer to specify which 10-15 files are most important for understanding the auth system.
- ☐ **D** Read all files containing "auth", "login", "permission", or "token" in their content or filename.

Your codebase exploration tool stores session IDs to allow engineers to continue investigations across work sessions. An engineer spent an hour yesterday analyzing a legacy authentication module, building context about its architecture and dependencies. They want to continue today. The session ID is valid, but version control shows 3 of the 12 files the agent previously read were modified overnight by a teammate's merge. What approach best balances efficiency and accuracy?

- ☐ **A** Resume the session and immediately have the agent re-read all 12 previously analyzed files
- ☒ **B** Resume the session and inform the agent which specific files changed for targeted re-analysis
- ☐ **C** Resume the session without informing the agent about the changed files
- ☐ **D** Start a fresh session to ensure the agent works with current codebase state without stale assumptions

Your code review assistant needs to analyze pull requests and provide feedback on three aspects: code style compliance, potential security issues, and documentation completeness. Each aspect requires reading files, running analysis tools, and generating a report section. The review process follows the same three-step workflow for every PR.

Which task decomposition pattern is most appropriate for this workflow?

- ☒ **A** Prompt chaining—break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step.
- ☐ **B** Single comprehensive prompt—include all instructions in one prompt and let the model handle all three aspects simultaneously.
- ☐ **C** Routing—classify each PR by type (feature, bugfix, refactor) first, then route to different review prompts optimized for that category.
- ☐ **D** Orchestrator-workers—have a central LLM analyze each PR to dynamically determine which checks are needed, then delegate to specialized worker LLMs for each identified subtask.

Your agent has spent 25 minutes exploring a game engine's rendering subsystem—reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering patterns" rather than the specific `VulkanPipeline` and `FrameGraph` classes it discovered earlier.

What's the most effective approach?

- ☐ **A** Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.
- ☐ **B** Continue in the current context with more targeted prompts referencing the specific classes by name.
- ☒ **C** Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context.
- ☐ **D** Use `/clear` to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.

After adding an MCP server with specialized code refactoring tools (`extract_function` , `rename_variable` , `inline_function`), you notice the agent still uses basic text manipulation via Write and Bash sed commands for refactoring tasks. The MCP server is connected and healthy. Examining the configuration, you find each MCP tool has a minimal description like "extract_function: Extracts a function from code."

What's the most effective way to improve adoption of the MCP refactoring tools?

- ☒ **A** Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs.
- ☐ **B** Implement a request classifier that detects refactoring intent and automatically routes those requests to the MCP server before the agent processes them.
- ☐ **C** Remove the Write tool from the agent's configuration for refactoring sessions so it must use the MCP tools for code modifications.
- ☐ **D** Accept this as expected behavior since simpler tools like sed are more predictable than specialized refactoring tools.

You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search. When a user asks the agent to "create a branch for JIRA-123 and add documentation links to the ticket," how does the agent access tools across these servers?

- ☐ **A** You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.
- ☒ **B** Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.
- ☐ **C** The agent automatically selects the most relevant server based on the request and loads only that server's tools.
- ☐ **D** The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.

Your agent has analyzed a complex service module—reading 23 source files, tracing request flows, and identifying error handling patterns. A developer wants to compare two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. They need to independently develop both approaches to evaluate trade-offs. How should you manage the sessions?

- ☐ **A** Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.
- ☐ **B** Start two fresh sessions, having each re-read the relevant source files before beginning.
- ☐ **C** Export the analysis session's key findings to a file, then create two new sessions that reference this file.
- ☒ **D** Resume the analysis session with `fork_session` enabled, creating a separate branch for each testing strategy.

Your order management system requires tools for three distinct operations: issuing refunds (requires amount and reason), canceling orders (requires reason), and requesting reshipments (requires shipping address). Each operation shares an `order_id` parameter but has different additional requirements. You notice during testing that with your current unified tool design, the agent frequently omits required parameters or includes irrelevant ones. What design change will most effectively improve parameter accuracy?

- ☒ **A** Split into three separate tools (`issue_refund` , `cancel_order` , `request_reshipment`), each defining only the parameters required for that specific operation.
- ☐ **B** Keep one unified tool with a nested `operation_details` object parameter whose internal structure varies by operation type, documented in the tool description.
- ☐ **C** Keep one unified tool with all parameters marked optional, but add detailed few-shot examples in the system prompt showing correct parameter combinations for each operation type.
- ☐ **D** Keep one unified tool but add JSON Schema if-then-else conditionals to enforce that parameters like `amount` are required only when the operation type is "refund".

Your `search_flights` tool calls an external airline API that occasionally returns a 503 Service Unavailable error. What is the most effective way to handle this error in your tool implementation?

- ☒ **A** Automatically retry the request up to five times with exponential backoff before returning results to the agent.
- ☐ **B** Log the error internally and return an empty response, letting the model continue without the flight data.
- ☐ **C** Return an error message in the tool result explaining the service is temporarily unavailable.
- ☐ **D** Return an empty flight list as if the search succeeded but found no matching flights.

Production monitoring shows your `search_catalog` tool fails 12% of the time: 8% are network timeouts that succeed when immediately retried, while 4% are query syntax errors from malformed user-provided filters that never succeed regardless of retry attempts. Currently, both error types are returned to the agent identically, causing it to waste turns retrying syntax errors and telling users to "try again later" for timeouts. How should you modify the tool's error handling?

- ☐ **A** Return all errors with a `retryable` boolean flag and error type details.
- ☐ **B** Apply exponential backoff retry logic to all errors uniformly, returning a generic "service temporarily unavailable" message after max retries are exhausted.
- ☒ **C** Implement automatic retry with backoff for network timeouts inside the tool; return syntax errors immediately with parameter validation details.
- ☐ **D** Add few-shot examples to your system prompt demonstrating how to distinguish network errors from syntax errors and handle each case appropriately.

Your marketing agent connects to MCP servers from three different vendors: an email service, a social media platform, and an analytics dashboard. Each server provides tool annotations—some tools marked with `readOnlyHint: true`, others with `destructiveHint: true`. Your team proposes automatically bypassing confirmation prompts for tools annotated as read-only to improve workflow speed. What should guide this decision?

- ☐ **A** Implement a verification layer that tests each tool's actual behavior before adding it to an allowlist. Skip confirmation only for tools validated through behavioral testing.
- ☐ **B** MCP servers run as local processes, so tools from any properly initialized connection inherit your application's security context and can be trusted regardless of vendor.
- ☐ **C** Read-only annotations indicate intended behavior and are reliable for tools that perform GET requests to external APIs. Skip confirmation for data-fetching tools from properly connected servers.
- ☒ **D** Treat tool annotations as untrusted metadata unless the server itself is trusted. Base confirmation requirements on your assessment of each vendor's trustworthiness, not on their self-reported annotations.

Your `control_device` tool manages smart home devices through external APIs. When a device doesn't respond within the timeout period, the tool returns an error. Production logs show that the agent simply tells users "the device is not responding" without offering helpful next steps. Which error response structure would best enable the agent to provide useful follow-up?

- ☐ **A** Set `is_error: false` with an optimistic message indicating the command was dispatched successfully but device acknowledgment is still pending.
- ☐ **B** Set `is_error: true` with a structured technical error containing the device ID, timeout duration, and raw API response code for debugging purposes.
- ☐ **C** Set `is_error: true` with a brief "Device offline" message and provide a separate tool the agent can call to retrieve context-specific troubleshooting suggestions.
- ☒ **D** Set `is_error: true` with a message explaining the likely cause and suggesting troubleshooting steps the agent can offer the user.

Your `track_shipment(tracking_id)` tool queries an external logistics API that sometimes fails—the API may be temporarily unavailable, the tracking ID may be malformed, or the shipment may not exist. Currently, your tool raises a Python exception when errors occur. Users report the agent gives unhelpful responses like “I’m having trouble with that request” instead of suggesting alternatives such as verifying the tracking number format or checking by order number. How should you handle errors in tool results?

- ☐ **A** Create dedicated error-recovery tools (`retry_tracking_lookup` , `search_by_order_number`) that the model can invoke after the primary tracking tool returns a failure indicator.
- ☐ **B** Implement retry logic with exponential backoff inside the tool implementation so transient errors are automatically handled and only return a result after all retry attempts are exhausted.
- ☒ **C** Return structured error information as normal tool output including error type, recoverability status, and actionable context for the user.
- ☐ **D** Return a generic error response (e.g., `{"success": false, "error": "lookup_failed"}`) for all failure cases to maintain a consistent schema and avoid exposing internal error details.

Your `update_user_profile` tool accepts a `user_id` (required) and an optional `fields_to_update` object. In testing, Claude frequently omits `user_id` or passes incorrectly structured data. What is most critical for helping Claude understand what parameter values to provide?

- ☐ **A** Detailed error responses explaining why invalid parameter values were rejected
- ☐ **B** Verbose parameter names encoding format hints, such as `user_id_string_uuid_format`
- ☒ **C** Strict JSON Schema type constraints marking `user_id` as required and defining `fields_to_update` as an object type
- ☐ **D** Clear parameter descriptions explaining expected format, such as "`user_id` : UUID of the user to update (required)"

Your `remove_team_member` tool uses a `dry_run: boolean` parameter for previewing impacts before execution. Production monitoring shows the agent bypasses the preview step in 15% of cases by calling with `dry_run=false` directly. You need to ensure every removal is preceded by a preview that the user explicitly confirms. What is the most reliable approach?

- ☐ **A** Annotate the tool as requiring confirmation and configure the orchestration layer to prompt the user for approval before forwarding any calls to annotated tools.
- ☐ **B** Add server-side validation that permits `dry_run=false` only when a `dry_run=true` call with identical parameters occurred within the past 60 seconds.
- ☐ **C** Add detailed instructions and few-shot examples to the tool description requiring the agent to always call with `dry_run=true` first and wait for user confirmation before calling with `dry_run=false`.
- ☒ **D** Replace with two tools: `preview_remove_member` returns impact details and a single-use confirmation token; `execute_remove_member` requires that token, binding execution to the specific previewed action.

Your `get_portfolio_value` tool returns the total value of a user's investment portfolio. You're deciding between returning a structured JSON object with explicit fields versus returning the information as a formatted text string. What is the primary advantage of using structured output with defined fields?

- ☐ **A** Structured JSON is processed deterministically by the model, significantly improving accuracy when extracting values.
- ☒ **B** The agent can reliably extract specific values without parsing free-form text, reducing errors in subsequent operations.
- ☐ **C** Structured JSON consumes significantly fewer tokens than natural language, substantially reducing API costs.
- ☐ **D** JSON schemas automatically validate that the underlying API returned correct data before the agent processes it.

Your `search_documents(query)` tool returns results as: "Found 3 documents: 'Q1 Budget Proposal', 'Q2 Budget Forecast', 'Annual Review'". You want the agent to chain this with action tools like `share_document(document_id, email)` and `move_document(document_id, folder)`. What return format would best enable these multi-step workflows?

- ☒ **A** Structured data containing document IDs and metadata for each result
- ☐ **B** URLs that users can click to access documents in their browser
- ☐ **C** A JSON array of document titles extracted from the search results
- ☐ **D** More detailed human-readable descriptions including file sizes and authors

Your `search_documents` tool needs a parameter to specify which database to search. Your organization has three document databases: "research_papers", "internal_reports", and "technical_specs". Users express this naturally in conversation ("search the research database", "check technical documents"). How should you design the database selection parameter?

- ☐ **A** A freeform string parameter with runtime validation that returns an error if the value doesn't match a known database.
- ☐ **B** A freeform string parameter where the backend uses semantic matching to determine which database(s) to search.
- ☒ **C** An enum parameter with values ["research_papers", "internal_reports", "technical_specs"], requiring the model to map natural language to the appropriate value.
- ☐ **D** No explicit parameter—search all three databases by default, then have the model filter results by source.

Your CRM agent's `delete_contact` tool handles requests like "delete the duplicate entry for Acme Corp." The database contains similarly named records (e.g., "Acme Corp," "Acme Corporation," "ACME Corp Inc."), and analytics show 8% of deletions are reversed within 24 hours due to misidentified records. Users have also complained that the current multi-step confirmation flow adds too much friction to routine cleanup tasks. Which approach most effectively reduces the error rate while maintaining workflow efficiency?

- ☐ **A** Deploy automated duplicate detection that identifies and merges probable duplicates, removing the need for manual deletion requests.
- ☐ **B** Require users to supply the exact record ID from the CRM interface rather than using natural language references to contact names.
- ☐ **C** Implement soft-delete with a 30-day recovery window so users can undo mistakes without slowing down the deletion workflow.
- ☒ **D** Present matched records with differentiating fields and require single-click confirmation of the intended target before executing deletion.

Your agent has access to 50+ specialized API connectors for different external services. As the connector library grew, tool selection accuracy dropped to 58%. You design a `search_connectors(description)` tool that finds matching connectors, but in testing, agents frequently skip searching and call connectors directly (often incorrectly), or search correctly but still select wrong connectors from the filtered results.

How should you design the tool composition pattern to address both issues?

- ☐ **A** Design connectors with built-in compatibility validation that returns descriptive errors for mismatched requests.
- ☐ **B** Design a `find_and_execute(description, params)` composite tool that searches and immediately executes the best-matching connector.
- ☐ **C** Enhance all connector descriptions with detailed usage examples, edge cases, and input requirements. Add few-shot examples showing the correct search-then-use workflow.
- ☒ **D** Design `search_connectors` to dynamically add matched connectors to the agent's available tools. Connectors start unavailable and persist once discovered.

Your `send_notification` tool calls third-party messaging APIs. When these services time out during delivery, you cannot determine whether the message was actually sent. Currently, the tool returns `is_error: true` with a generic "Notification failed" message for all timeouts. Production monitoring reveals agents automatically retry these failures, frequently causing users to receive duplicate notifications. How should you modify the error response?

- ☐ **A** Return `is_error: true` with a message encouraging retry: "Delivery service temporarily unavailable. Please retry the notification."
- ☐ **B** Return `is_error: true` with a structured field `retry_safe: true` for timeouts, distinguishing them from permanent failures that should not be retried.
- ☒ **C** Return `is_error: true` with a message communicating uncertainty: "Timeout—status unknown. Message may have been sent. Avoid retry."
- ☐ **D** Return `is_error: false` with the original message content echoed back.

Your MCP server implements a `check_availability` tool that queries an external calendar API. During testing, you encounter three error conditions: (1) the tool is called with a malformed request missing the required `user_email` parameter, (2) the calendar API returns a 404 because the specified user doesn't exist in the calendar system, and (3) the calendar API returns a 503 because the service is temporarily unavailable. How should each error be reported according to MCP's error handling design?

- ☐ **A** Report all three as JSON-RPC protocol errors.
- ☒ **B** Report errors 1 and 2 as JSON-RPC protocol errors; report error 3 as a tool result with `isError: true`.
- ☐ **C** Report error 1 as a JSON-RPC protocol error; report errors 2 and 3 as tool results with `isError: true`.
- ☐ **D** Report all three as tool results with `isError: true`.

Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

- ☐ **A** Submit all documents to the Batch API with `custom_ids` for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.
- ☐ **B** Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.
- ☐ **C** Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.
- ☒ **D** Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

After implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats—inline citations vs. bibliographies, methodology sections vs. details embedded in introductions. What's the most effective way to address these failures?

- ☐ **A** Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation.
- ☐ **B** Implement retry logic that re-sends requests when validation detects empty required fields.
- ☐ **C** Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract.
- ☒ **D** Add few-shot examples demonstrating extractions from documents with varied structures—showing how to identify citations in different formats and locate methodology details across section types.

Monitoring reveals that specifications sometimes appear inconsistently within source documents. For example, a summary section might state "Battery: 4000mAh" while the detailed specs table shows "Battery: 4200mAh." Your current schema uses a single `battery_capacity_mah` field. This inconsistency occurs in approximately 15% of documents, and historical analysis confirms the detailed specs table is more accurate 90% of the time. What's the most effective approach?

- ☒ **A** Change to an array field that captures all values found with their source locations, letting downstream systems apply precedence rules.
- ☐ **B** Include extraction instructions specifying to prefer values from the detailed specs table when multiple values exist, keeping the single-value schema.
- ☐ **C** Implement schema validation that rejects results containing conflicting values, requiring source document correction before processing.
- ☐ **D** Add a `conflict_detected` boolean field that flags inconsistencies, triggering manual review for affected documents.

Your team is extracting structured data from 50,000 legacy legal contracts under a two-week deadline. Initial testing with 500 sample documents shows 82% pass JSON schema validation on the first attempt, while the remaining 18% fail due to diverse issues—missing required fields, malformed dates, and incorrectly identified parties. Documents that fail typically need 2-3 prompt refinements targeting their specific failure modes before extraction succeeds. Which batch processing strategy is the most cost-efficient while still meeting the deadline?

- ☐ **A** Submit all 50,000 documents via batch API, then submit failed extractions in successive batches—refining prompts between each batch—until all documents pass validation.
- ☐ **B** Split documents into 10 sequential batches of 5,000 each, analyzing results and refining prompts between batches to improve extraction quality progressively.
- ☐ **C** Use the real-time API for all 50,000 documents since the batch API's 24-hour processing window creates unacceptable deadline risk.
- ☒ **D** Process 2,000 sample documents via real-time API to identify failure patterns and refine prompts, then batch process all 50,000 with the optimized prompts.

Your extraction pipeline validates outputs against JSON schemas, but you need to implement human review given limited reviewer capacity (they can handle approximately 5% of total extraction volume). What's the most effective basis for selecting which extractions to route for human review?

- ☐ **A** Route extractions containing specific high-priority entity types (e.g., financial figures, dates) for human review, regardless of extraction confidence.
- ☐ **B** Randomly sample 5% of extractions for review.
- ☒ **C** Route extractions where the model indicates low confidence or where source documents contain ambiguous or contradictory information.
- ☐ **D** Route extractions for review only when downstream systems report data quality issues or processing failures.

After your daily batch of 10,000 documents completes, 300 documents (3%) failed with "context_length_exceeded" errors. The results file identifies each failure by custom_id. What's the most cost-effective approach to process these failures?

- ☐ **A** Increase the max_tokens parameter for the 300 failed documents and resubmit them in a new batch
- ☐ **B** Resubmit the entire 10,000 document batch using a model tier with a larger context window
- ☐ **C** Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts
- ☒ **D** Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions

The system processes product reviews using tool use with a defined schema: rating (integer 1-5), pros (string array), cons (string array), and overall_sentiment (enum: positive, negative, mixed). Testing reveals two issues with brief or ambiguous reviews (~20% of the dataset): (1) for reviews like "Great product!", Claude fabricates specific pros and cons rather than indicating this information isn't explicitly stated, and (2) for sarcastic reviews like "Well that was... interesting", Claude picks sentiment arbitrarily since there's no option for ambiguous cases. What schema modification best addresses both issues?

- ☐ **A** Add an extraction_confidence field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold.
- ☐ **B** Allow null values for pros/cons, and add "unclear" to the sentiment enum.
- ☐ **C** Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum.
- ☒ **D** Allow empty arrays for pros/cons as valid output, and add "unclear" to the sentiment enum.

Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned." What's the most effective improvement?

- ☐ **A** Enrich the schema to `{skill: string, confidence: float, source_quote: string}[]` to capture extraction metadata.
- ☐ **B** Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."
- ☐ **C** Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.
- ☒ **D** Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

Your extraction pipeline processes contracts that frequently include amendments. When a contract contains both original terms and later amendments (e.g., original clause specifies "30-day payment terms" while Amendment 1 changes this to "45 days"), the model inconsistently extracts one value or the other with no indication of which applies. What's the most effective approach to improve extraction accuracy for documents with amendments?

- ☐ **A** Add prompt instructions to always extract the most recent amendment value and ignore superseded original terms.
- ☒ **B** Redesign the schema so amended fields capture multiple values, each with source location and effective date.
- ☐ **C** Implement post-extraction validation using pattern matching to detect amendments and flag those extractions for manual review.
- ☐ **D** Preprocess documents with a classifier that identifies and removes superseded sections before the main extraction step.

Your extraction tool uses a JSON schema where `property_type` is defined as an enum: `['house', 'apartment', 'condo', 'townhouse']`. After deployment, 8% of extractions fail schema validation. Investigation reveals listings mention many uncommon property types—"studio", "loft", "duplex", "mobile home", "tiny house", "converted warehouse"—and new types continue appearing regularly. What's the most effective long-term solution?

- ☐ A Add an "other" value to your enum with a separate `property_type_detail` string field for specifics when "other" is selected.
- ☐ B Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value.
- ☒ C Change `property_type` from an enum to a free-form string and implement a normalization step in post-processing.
- ☐ D Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases.

Your system has been running for 3 weeks and human reviewers have corrected 847 extractions. Analysis reveals a recurring pattern: when recipes use informal measurements like "a handful" or "a splash," the model either invents specific amounts or leaves fields empty—accounting for 23% of all corrections. How should you use this feedback to improve extraction accuracy?

- ☐ **A** Update your JSON schema to add a "measurement_type" enum field (precise/informal).
- ☐ **B** Fine-tune the model on the 847 corrected extractions.
- ☐ **C** Implement a post-processing layer that uses pattern matching to detect informal measurement phrases in source text and automatically populate values when the extraction is empty.
- ☒ **D** Add few-shot examples to your prompt demonstrating correct handling of informal measurements—extracting them verbatim rather than converting or omitting them.

After deployment, you find that 12% of extractions contain semantic errors that pass JSON schema validation (e.g., a duration like "30 minutes" incorrectly placed in an ingredient quantity field). Human reviewers have capacity to check only 20% of extractions. Which approach most effectively allocates reviewer attention?

- ☐ **A** Randomly sample 20% of extractions for review, using corrections to track accuracy and identify error patterns.
- ☒ **B** Have the model output field-level confidence scores, then calibrate review thresholds using a labeled validation set.
- ☐ **C** Prioritize review of all extractions where required fields are empty or explicitly marked as not found.
- ☐ **D** Review all extractions from documents with formatting anomalies such as unusual layouts or mixed content types.

Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence $\geq 90\%$ have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions. Before deploying, what validation step is most critical?

- ☐ **A** Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.
- ☐ **B** Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.
- ☒ **C** Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.
- ☐ **D** Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.

Your extraction system uses tool_use with a JSON schema containing 12 fields and detailed descriptions, totaling approximately 2,500 tokens for the complete tool definition. Processing documents under 150K tokens yields 98% accuracy. For documents between 175-190K tokens, accuracy drops to 71%, with information from the final third consistently missed. The model's context window is 200K tokens. What is the most likely cause?

- ☐ **A** Very long documents exceed the model's effective attention span regardless of context limits, causing accuracy degradation for content farther from the prompt instructions.
- ☐ **B** Schemas exceeding 8-10 fields increase decision complexity during parameter generation, reducing extraction accuracy independent of document length.
- ☐ **C** The model distributes attention proportionally across input length, causing fields mentioned only once near the document's end to receive insufficient processing focus.
- ☒ **D** Tool definitions consume input context tokens. Combined with system prompts and document content, the total approaches the context limit, degrading end-of-document processing.

Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article—for example, outputting "500" for attendee_count when the source contains no attendance information. What's the most effective way to reduce these false extractions?

- ☐ **A** Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies.
- ☐ **B** Add a post-processing step using a second LLM call to verify each extracted value exists in the source document.
- ☐ **C** Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data.
- ☒ **D** Add prompt instructions to return null for any field where information is not directly stated in the source.

When the agent calls `lookup_order` and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call `process_refund` or `escalate_to_human` next?

- ☐ A The agent executes the remaining steps in a tool sequence planned at the start of the request.
- ☐ B The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.
- ☐ C The orchestration layer automatically routes to the next tool based on the order's status field.
- ☒ D The order details are added to the conversation and the model reasons about which action to take.

Compliance requires that refunds exceeding \$500 must automatically escalate to a human agent—this rule cannot be left to model discretion. Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate). How should you achieve guaranteed compliance?

- ☐ **A** Modify the refund tool to return an error with message "Amount exceeds policy limit—please escalate" when threshold is exceeded.
- ☐ **B** Strengthen the system prompt with emphatic language: "CRITICAL POLICY: Refunds over \$500 MUST trigger human escalation. NEVER process these directly."
- ☒ **C** Implement a hook to intercept tool calls; when the refund process amount exceeds \$500, block it and invoke human escalation.
- ☐ **D** Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts (\$400, \$500, \$600).

Your agent has called `lookup_order` multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment info, status history). Tool outputs now represent the majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the most effective approach before making additional lookups?

- ☐ **A** Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues
- ☐ **B** Proceed with additional lookups without modifying the existing tool output context
- ☒ **C** Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details
- ☐ **D** Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions

Your `process_refund` tool returns two types of errors: technical errors ("503 Service Unavailable", "Connection timeout") that are transient (5% of calls), and business errors ("Order exceeds 30-day return window", "Item already refunded") that are permanent (12% of calls). Monitoring shows the agent wastes 3-4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude.

What's the most effective way to reduce wasted retries while improving customer-facing response quality?

- ☐ **A** Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries.
- ☐ **B** Add few-shot examples showing how to distinguish retrieable from non-retrieable errors by parsing error message text.
- ☐ **C** Add a `check_refund_eligibility` tool that must be called before `process_refund` to prevent business rule violations.
- ☒ **D** Return structured error responses with `retrieable: false` for business errors and a customer-friendly explanation for Claude to use.

Your agent is handling a billing dispute. After calling `get_customer` and `lookup_order`, it identifies that the dispute involves a promotional pricing error requiring manager approval—beyond the agent's authorization level. How should the workflow handle this mid-process escalation?

- ☐ **A** Persist the complete conversation and tool response history to a database, then call `escalate_to_human` with a reference ID.
- ☒ **B** Compile a structured handoff with customer details, order info, and the identified issue before calling `escalate_to_human`.
- ☐ **C** Attempt the refund with `process_refund` anyway, escalating only if the system rejects the transaction.
- ☐ **D** Call `escalate_to_human` passing only the customer's original message.

A customer raises three separate issues during one session: a refund inquiry (turns 1-15), a subscription question (turns 16-30), and a payment method update (turns 31-45). At turn 48, the customer asks "What happened with my refund?" The conversation is approaching context limits. What strategy best maintains the agent's ability to address all issues throughout the session?

- ☒ **A** Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer.
- ☐ **B** Summarize earlier turns into a narrative description, preserving full message history only for the active issue.
- ☐ **C** Implement sliding window context that retains the most recent 30 turns.
- ☐ **D** Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues.

A customer writes: "I've been going back and forth on this return for days. I just want to speak to someone who can actually help me." The agent has confirmed via `lookup_order` that the return is straightforward—within policy and eligible for immediate processing. What should the agent do?

- ☐ **A** Ask what specifically hasn't worked in previous attempts before deciding whether to escalate or resolve automatically
- ☐ **B** Process the refund via `process_refund` to resolve the underlying issue, then inform them it's complete
- ☒ **C** Acknowledge frustration, inform them this is resolvable now, and offer to complete it or escalate
- ☐ **D** Call `escalate_to_human` immediately to honor the customer's request

When implementing your `lookup_order` MCP tool, the backend sometimes returns errors (e.g., "Order not found" or temporary database failures). What is the correct pattern for communicating these errors back to the agent?

- ☒ **A** Return the error message in the tool result content with the `isError` flag set to true
- ☐ **B** Log the error server-side and return an empty result to avoid confusing the model
- ☐ **C** Throw an exception from the tool handler so the agent framework can catch and log it
- ☐ **D** Return a success response with a "status" field indicating the error type

After investigating a billing dispute over 25+ turns, you've identified that duplicate charges occurred due to a payment gateway timeout triggering retry logic. The required refund (\$847) exceeds your \$500 authorization limit. You need to call `escalate_to_human`, and the human agent won't have access to your conversation transcript. What context should you pass to enable effective resolution?

- ☐ **A** The customer's original complaint verbatim plus the tool result excerpts showing duplicate transactions.
- ☐ **B** The complete conversation transcript with all tool results.
- ☒ **C** A structured summary: customer ID, root cause, refund amount, and recommended action.
- ☐ **D** Your diagnosis and the refund amount only.

The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened. What's the most likely cause of this behavior?

- ☐ **A** The prompt lacks instructions telling Claude to remember information across multiple exchanges.
- ☐ **B** Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.
- ☒ **C** The conversation history isn't being passed in subsequent API requests.
- ☐ **D** The verification tool is clearing the agent's internal state after each successful validation step.

A customer sends: "This is frustrating. I've explained my issue twice and nothing is being resolved. I want to talk to a real person NOW." The agent has not yet called any tools to investigate their account. What should the agent do?

- ☒ **A** Acknowledge the frustration and ask one targeted question to understand the specific issue before escalating.
- ☐ **B** First call `get_customer` and `lookup_order` to gather account context, then escalate to a human agent.
- ☐ **C** Immediately call `escalate_to_human` with the conversation history.
- ☐ **D** Briefly explain what the agent can help with and offer to resolve the issue quickly, escalating only if the customer repeats their request.

You're implementing the escalation logic for when the agent should call `escalate_to_human`. Your team proposes four different approaches for triggering escalation. Which approach will most reliably identify cases that genuinely require human intervention?

- ☐ **A** Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions, removing the need for model judgment calls.
- ☐ **B** Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human.
- ☒ **C** Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.
- ☐ **D** Implement sentiment analysis that monitors for frustration indicators (negative language, repeated questions, exclamation marks) and trigger escalation when the frustration score exceeds a configured threshold.

A customer returns 4 hours after their initial session about the same billing dispute. The previous 32-turn session contains `lookup_order` results showing "Status: PENDING, Expected resolution: 24-48 hours." In testing, you observe that when resuming sessions with stale tool results, the agent often references the outdated data in responses (e.g., "I see your refund is still being processed") even after subsequent fresh tool calls return different information. What approach most reliably handles returning customers?

- ☐ **A** Resume with full history but filter out previous `tool_result` messages before resuming, keeping only the human/assistant turns so the agent must re-fetch needed data.
- ☒ **B** Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging.
- ☐ **C** Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to the same tool exist in context.
- ☐ **D** Resume with full history and configure the agent to automatically re-call all previously-used tools at session start to ensure data freshness.

During a billing dispute resolution, your agent successfully retrieves customer info via `get_customer` and order details via `lookup_order`, but when attempting to call `process_refund`, the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot actually process the refund due to the backend failure. What approach best balances first-contact resolution with appropriate error handling?

- ☐ **A** Escalate immediately to a human agent since the refund action cannot be completed
- ☒ **B** Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later
- ☐ **C** Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically
- ☐ **D** Implement automatic retries with exponential backoff for `process_refund`, keeping the conversation open until the refund is successfully processed

Production logs reveal inconsistent error handling: when `lookup_order` fails, the agent sometimes retries 5+ times (wasteful when the order ID doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses: `{"isError": true, "content": [{"type": "text", "text": "Operation failed"}]}`. The agent cannot distinguish between error types. What's the most effective improvement?

- ☒ **A** Enhance error responses with structured metadata: include `errorCategory` (transient/validation/permission), `isRetryable` boolean, and a description of what caused the failure.
- ☐ **B** Create an `analyze_error` MCP tool the agent calls after any failure to determine the error category and recommended action.
- ☐ **C** Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.
- ☐ **D** Add few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each.